

Callable Bond Pricer

Two-Factor Hull-White (G2++) Methodology,
Mathematical Derivation, and Longstaff-Schwartz Implementation

Aegis Quant Library — `MCPricer.Bonds`

August 19, 2026

Contents

1	Introduction and Scope	3
1.1	Product being priced	3
2	The Two-Factor Hull-White (G2++) Short-Rate Model	3
2.1	Model dynamics and measure	3
2.2	Explicit solution of the OU factors	4
2.3	Exact discretization for simulation	4
3	Fitting the Initial Term Structure	4
3.1	Bond price as a conditional expectation	4
3.2	The conditional mean: the function $B(a, \tau)$	5
3.3	The conditional variance: the kernel $V(\tau)$	5
3.4	The deterministic shift $\varphi(t)$	5
3.5	Closed-form bond price $P(t, T)$	6
4	European Option on a Zero-Coupon Bond (ZBO)	6
4.1	Setup	7
4.2	Change to the t -forward measure	7
4.3	The variance $\Sigma^2(t, T)$	7
4.4	Closed-form price	7
4.5	Edge cases and identities	8
5	Calibration Methodology	8
5.1	Calibration instruments	8
5.2	Objective function	8
5.3	Unconstrained reparameterization	8
5.4	Nelder-Mead simplex minimization	8
5.5	Overfitting guard and failure handling	9
5.6	Round-trip validation	9
6	Monte Carlo Simulation of the Model	9
6.1	Pricing identity	9
6.2	Fine grid and φ precomputation	9
6.3	Discounting via trapezoidal integration	10
6.4	Correlated increments	10
7	The Callable Bond Product	10

7.1	Cashflow schedule	10
7.2	Call schedule economics	10
7.3	Merged event grid	10
8	Longstaff-Schwartz Valuation of the Embedded Call	11
8.1	Dynamic-programming formulation	11
8.2	Why regression: avoiding look-ahead bias	11
8.3	Basis functions	11
8.4	Reformulation in PV_0 units: the implemented algorithm	11
9	Numerical Implementation Notes	12
9.1	Memory layout	12
9.2	Parallelism	12
9.3	DV01	12
10	Validation and Benchmarks	13
11	Known Limitations	13
12	Code-to-Formula Mapping	14
	References	14

1 Introduction and Scope

This document is the complete mathematical and methodological reference for the callable bond pricer implemented in `MCPricers/Bonds`. It covers, in full derivation:

- the two-factor Hull-White (“G2++”) short-rate model of Brigo & Mercurio [\[\[1\]\]](#), including how it is fitted to an arbitrary input discount curve;
- the closed-form zero-coupon bond price $P(t, T)$ and the closed-form European option on a zero-coupon bond (ZBO), used both as the calibration pricing formula and as an analytical benchmark;
- the nonlinear least-squares calibration procedure that fits the five free model parameters to market ZBO prices;
- the Monte Carlo simulation scheme (exact factor transition, trapezoidal short-rate discounting);
- the Longstaff & Schwartz [\[\[2\]\]](#) regression-based valuation of the issuer’s early-redemption (call) option; and
- the validation methodology and known numerical limitations.

Every formula below is cross-referenced to the C# source file and method that implements it (Section 12 collects the full map). The implementation lives entirely in namespace `MCPricers.Bonds`: model (`G2ppParameters.cs`, `G2ppAnalytics.cs`), calibration (`G2ppCalibrator.cs`, `NelderMead.cs`), and the product and pricer (`CallableBondDefinition.cs`, `CallableBondMCPricer.cs`), with tests in `MCPricers.Tests/Bonds`.

1.1 Product being priced

A *callable bond* pays fixed coupons $c \cdot N/m$ at m dates per year until maturity T_M , at which point the principal N is redeemed. In addition, the issuer holds the right, at each date in a pre-specified *call schedule* $\{t_1^c, \dots, t_n^c\} \subset (0, T_M)$, to redeem the bond early by paying $K_i \cdot N$ (the call price, a fraction of par) instead of the remaining cashflows. This is a Bermudan-style option against the bondholder: it can only reduce the bondholder’s realized value relative to the equivalent non-callable (“bullet”) bond, never increase it.

2 The Two-Factor Hull-White (G2++) Short-Rate Model

2.1 Model dynamics and measure

Under the risk-neutral measure $\mathbb{Q}$ (numeraire: the money-market account $B(t) = \exp(\int_0^t r(s) ds)$), the short rate is decomposed into two correlated Ornstein-Uhlenbeck (OU) factors plus a deterministic shift:

$$dx(t) = -a x(t) dt + \sigma dW_1(t), \quad x(0) = 0, \quad (1)$$

$$dy(t) = -b y(t) dt + \eta dW_2(t), \quad y(0) = 0, \quad (2)$$

$$dW_1(t) dW_2(t) = \rho dt, \quad (3)$$

$$r(t) = x(t) + y(t) + \varphi(t). \quad (4)$$

Here $a, b > 0$ are mean-reversion speeds, $\sigma, \eta \geq 0$ are factor volatilities, $\rho \in (-1, 1)$ is the instantaneous correlation between the two Brownian drivers, and $\varphi(t)$ is a deterministic function chosen so that the model reproduces an arbitrary input discount curve exactly (derived in Section 3). In code, the five free parameters $(a, \sigma, b, \eta, \rho)$ are the immutable `G2ppParameters` record (`A1`, `Sigma1`, `A2`, `Sigma2`, `Rho`).

Remark 2.1 (Why two factors). A one-factor Hull-White model produces perfectly correlated changes at every point of the yield curve (all forward rates move together, scaled by $B(a, \tau)$).

Two imperfectly correlated factors ($\rho < 1$) let short-end and long-end rate volatility decouple, which a single factor cannot reproduce – essential for calibrating simultaneously to short- and long-dated cap/swaption volatilities.

2.2 Explicit solution of the OU factors

Equation (1) is a linear SDE; multiplying by the integrating factor e^{at} and integrating gives the explicit (“exact”) solution

$$x(t) = x(0)e^{-at} + \sigma \int_0^t e^{-a(t-u)} dW_1(u), \quad (5)$$

and analogously for $y(t)$ with (b, η, W_2) . Because the stochastic integral in (5) is a Wiener integral of a deterministic integrand, $x(t)$ is Gaussian conditional on $\mathcal{F}_0$, with

$$\mathbb{E}[x(t)] = x(0)e^{-at}, \quad \text{Var}[x(t)] = \sigma^2 \int_0^t e^{-2a(t-u)} du = \frac{\sigma^2}{2a} (1 - e^{-2at}).$$

By the same Itô-isometry argument applied jointly to x and y (using $dW_1 dW_2 = \rho dt$),

$$\text{Cov}[x(t), y(t)] = \frac{\rho\sigma\eta}{a+b} (1 - e^{-(a+b)t}). \quad (6)$$

2.3 Exact discretization for simulation

Equation (5) gives the exact transition law over a step $[t, t + \Delta]$: applying it with the lower integration limit shifted from 0 to t ,

$$x(t + \Delta) = x(t)e^{-a\Delta} + \sigma \sqrt{\frac{1 - e^{-2a\Delta}}{2a}} Z_1, \quad Z_1 \sim \mathcal{N}(0, 1), \quad (7)$$

and correspondingly for y with $Z_2 \sim \mathcal{N}(0, 1)$, $\text{Corr}(Z_1, Z_2) = \rho$. Because (7) is the *exact* conditional law (not a first-order Euler approximation), simulating (x, y) this way introduces *zero* time-discretization bias regardless of step size Δ – the only simulation error in the engine comes from the trapezoidal integration of $r(t)$ used for discounting (Section 6.3), not from the factor dynamics themselves. This is implemented in `CallableBondMCPricer.SimulateForward` as `decay1 = exp(-A1*dt)`, `vol1 = Sigma1*sqrt((1-exp(-2*A1*dt))/(2*A1))`, consuming the correlated (Z_1, Z_2) pair directly from `RandomSimulator.SimulationCube`, whose built-in Cholesky factorization of the 2×2 correlation matrix $\begin{bmatrix} 1 & \rho \\ \rho & 1 \end{bmatrix}$ supplies exactly the correlation required by (3).

3 Fitting the Initial Term Structure

3.1 Bond price as a conditional expectation

By definition, the time- t price of a zero-coupon bond maturing at T is

$$P(t, T) = \mathbb{E}^{\mathbb{Q}} \left[\exp \left(- \int_t^T r(s) ds \right) \middle| \mathcal{F}_t \right]. \quad (8)$$

Since $r(s) = x(s) + y(s) + \varphi(s)$ and φ is deterministic, and since x, y are jointly Gaussian conditional on $\mathcal{F}_t$ (Markov property of the OU process), the exponent’s integral $I := \int_t^T r(s) ds$ is Gaussian conditional on $\mathcal{F}_t$. Using $\mathbb{E}[e^{-I}] = \exp(-\mathbb{E}[I] + \frac{1}{2}\text{Var}[I])$ for a Gaussian I ,

$$P(t, T) = \exp \left(- \int_t^T \varphi(s) ds - \mathbb{E} \left[\int_t^T (x(s) + y(s)) ds \middle| \mathcal{F}_t \right] + \frac{1}{2} \text{Var} \left[\int_t^T (x(s) + y(s)) ds \middle| \mathcal{F}_t \right] \right). \quad (9)$$

3.2 The conditional mean: the function $B(a, \tau)$

Define $B(a, \tau) := \frac{1 - e^{-a\tau}}{a}$. From $\mathbb{E}[x(s)|\mathcal{F}_t] = x(t)e^{-a(s-t)}$,

$$\mathbb{E}\left[\int_t^T x(s) ds \mid \mathcal{F}_t\right] = x(t) \int_t^T e^{-a(s-t)} ds = x(t) B(a, T-t),$$

and analogously $\mathbb{E}[\int_t^T y ds | \mathcal{F}_t] = y(t)B(b, T-t)$.

3.3 The conditional variance: the kernel $V(\tau)$

Write $\tau = T - t$. Substituting the exact solution (5) (restarted at t) into the integral and swapping the order of integration (Fubini):

$$\int_t^T x(s) ds = x(t)B(a, \tau) + \sigma \int_t^T \int_t^s e^{-a(s-u)} dW_1(u) ds = x(t)B(a, \tau) + \sigma \int_t^T B(a, T-u) dW_1(u),$$

using $\int_u^T e^{-a(s-u)} ds = B(a, T-u)$. The stochastic part is a Wiener integral, so by the Itô isometry (jointly for x and y , using $dW_1 dW_2 = \rho dt$):

$$\text{Var}\left[\int_t^T (x(s)+y(s))ds\right] = \sigma^2 \int_t^T B(a, T-u)^2 du + \eta^2 \int_t^T B(b, T-u)^2 du + 2\rho\sigma\eta \int_t^T B(a, T-u)B(b, T-u)du. \quad (10)$$

Substituting $v = T - u$ (so v ranges over $[0, \tau]$), the three integrals in (10) depend only on τ – not on t and T separately – because the OU coefficients are time-homogeneous and $x(0) = y(0) = 0$. Evaluating each in closed form:

$$\begin{aligned} \int_0^\tau B(a, v)^2 dv &= \frac{1}{a^2} \int_0^\tau (1 - e^{-av})^2 dv = \frac{1}{a^2} \left[\tau + \frac{2}{a}e^{-a\tau} - \frac{1}{2a}e^{-2a\tau} - \frac{3}{2a} \right], \\ \int_0^\tau B(a, v)B(b, v) dv &= \frac{1}{ab} \int_0^\tau (1 - e^{-av})(1 - e^{-bv}) dv = \frac{1}{ab} \left[\tau + \frac{e^{-a\tau} - 1}{a} + \frac{e^{-b\tau} - 1}{b} - \frac{e^{-(a+b)\tau} - 1}{a+b} \right]. \end{aligned}$$

Collecting terms, define the (time-homogeneous) *variance kernel*

$$\begin{aligned} V(\tau) &:= \frac{\sigma^2}{a^2} \left[\tau + \frac{2}{a}e^{-a\tau} - \frac{1}{2a}e^{-2a\tau} - \frac{3}{2a} \right] + \frac{\eta^2}{b^2} \left[\tau + \frac{2}{b}e^{-b\tau} - \frac{1}{2b}e^{-2b\tau} - \frac{3}{2b} \right] \\ &\quad + \frac{2\rho\sigma\eta}{ab} \left[\tau + \frac{e^{-a\tau} - 1}{a} + \frac{e^{-b\tau} - 1}{b} - \frac{e^{-(a+b)\tau} - 1}{a+b} \right]. \end{aligned} \quad (11)$$

so that $\text{Var}[\int_t^T (x+y)ds | \mathcal{F}_t] = V(T-t)$, independent of the conditioning state. One checks directly that $V(0) = 0$ (every bracket vanishes at $\tau = 0$), consistent with $P(t, t) = 1$. Code: [G2ppAnalytics.VFunction](#) (private) implements (11) verbatim.

3.4 The deterministic shift $\varphi(t)$

Substituting the mean and variance above into (9) with $t = 0$, $x(0) = y(0) = 0$:

$$P(0, T) = \exp\left(-\int_0^T \varphi(s) ds + \frac{1}{2}V(T)\right). \quad (12)$$

No-arbitrage requires $P(0, T) = P^M(0, T)$ (the input market curve) for *every* T . Solving (12) for the integral and differentiating both sides with respect to T (Leibniz rule) gives φ pointwise:

$$\int_0^T \varphi(s) ds = -\ln P^M(0, T) + \frac{1}{2}V(T) \implies \varphi(T) = f^M(0, T) + \frac{1}{2} \frac{dV}{dT}(T), \quad (13)$$

where $f^M(0, T) = -\partial_T \ln P^M(0, T)$ is the market instantaneous forward rate. Differentiating (11) term by term (each bracket's derivative telescopes; e.g. $\frac{d}{dT} [\frac{2}{a} e^{-aT} - \frac{1}{2a} e^{-2aT}] = -2e^{-aT} + e^{-2aT}$, and adding the constant term's derivative 1 gives $(1 - e^{-aT})^2$):

$$\frac{dV}{dT}(T) = \sigma^2 B(a, T)^2 + \eta^2 B(b, T)^2 + 2\rho\sigma\eta B(a, T)B(b, T).$$

Substituting into (13) yields the closed form used by the code:

$$\varphi(t) = f^M(0, t) + \frac{\sigma^2}{2a^2} (1 - e^{-at})^2 + \frac{\eta^2}{2b^2} (1 - e^{-bt})^2 + \frac{\rho\sigma\eta}{ab} (1 - e^{-at})(1 - e^{-bt}) \quad (14)$$

This is the *unique* deterministic shift making the model reprice the input curve exactly, for *any* choice of $(a, \sigma, b, \eta, \rho)$ – curve-fitting and volatility calibration are fully decoupled. Code: [G2ppAnalytics.Phi](#).

Remark 3.1 (Instantaneous forward via finite difference). $f^M(0, t)$ is not available in closed form from a piecewise-linear zero-rate curve; [G2ppAnalytics.InstantaneousForward](#) approximates it by a centered finite difference of [ZeroCurve.ForwardRate](#) over a window $[t - h, t + h]$ with $h = 10^{-4}$ years (≈ 53 minutes) – far finer than any realistic curve pillar spacing, so the approximation error is negligible next to curve interpolation error itself.

3.5 Closed-form bond price $P(t, T)$

Combining Sections 3.3–3.4: from (13),

$$\int_t^T \varphi(s) ds = \int_0^T \varphi ds - \int_0^t \varphi ds = \ln \frac{P^M(0, t)}{P^M(0, T)} + \frac{1}{2} (V(T) - V(t)),$$

(here $V(T) := V(0, T)$, $V(t) := V(0, t)$ by the time-homogeneity noted above). Substituting into (9) together with the mean/variance results:

$$P(t, T) = \frac{P^M(0, T)}{P^M(0, t)} \exp\left(\frac{1}{2} (V(T-t) - V(T) + V(t)) - B(a, T-t)x(t) - B(b, T-t)y(t)\right) \quad (15)$$

Code: [G2ppAnalytics.ZeroCouponBondPrice](#).

Proposition 3.1 (Sanity checks). Formula (15) satisfies:

1. $P(0, T) = P^M(0, T)$ for $x(0) = y(0) = 0$ (the exponent reduces to $\frac{1}{2}(V(T) - V(T) + V(0)) = 0$ since $V(0) = 0$): the model reproduces the input curve *exactly*, independent of $(a, \sigma, b, \eta, \rho)$.
2. $P(t, t) = 1$ for any $x(t), y(t)$ (implemented as an explicit $\tau = 0$ short-circuit to avoid $0/0$ in $B(a, 0)$).
3. $\partial P / \partial x(t) = -B(a, T-t) P(t, T) < 0$ and likewise for y : a higher realized short-rate factor strictly lowers the bond price, for any $a, \tau > 0$.
4. $P(t, T)$ is strictly decreasing in T for fixed t (positive rates $\Rightarrow$ longer maturities discount more).

These four properties are exercised directly by [G2ppAnalyticsTests.cs](#) (`ZeroCouponBondPrice_AtT0_MatchesMarketCurveExactly`, `...SameMaturityAsValuation_IsOne`, `...PositiveStateShiftsDownwardBondPrice`, `...MonotonicallyDecreasingInMaturity`).

4 European Option on a Zero-Coupon Bond (ZBO)

The ZBO is the model's native calibration instrument: caps, floors, and (via standard approximations) swaptions all decompose into portfolios of options of this type, and – because the model is Gaussian – its price has the same closed form as a Black-76 option on a forward.

4.1 Setup

Consider a European call, expiring at t , on a zero-coupon bond of face value N maturing at $T > t$, struck at X :

$$\text{payoff at } t: \quad N \cdot (P(t, T) - X/N)^+ = (N P(t, T) - X)^+.$$

Its time-0 price is $\mathbb{E}^{\mathbb{Q}}[B(t)^{-1}(N P(t, T) - X)^+]$.

4.2 Change to the t -forward measure

Switch numeraire from the money-market account B to the t -maturity bond $P(\cdot, t)$, defining the t -forward measure $\mathbb{Q}^t$ (Section 2 of the companion theory document, [theory.tex](#), develops the change-of-numeraire theorem in general; here T there = t here). Then

$$\text{ZBC}(t, T, X, N) = P(0, t) \mathbb{E}^{\mathbb{Q}^t}[(N P(t, T) - X)^+].$$

Girsanov's theorem states that changing measure from $\mathbb{Q}$ to $\mathbb{Q}^t$ shifts the *drift* of $x(t), y(t)$ but leaves their *volatility structure* (quadratic variation) unchanged – $x(t), y(t)$ remain jointly Gaussian under $\mathbb{Q}^t$ with the *same* variance/covariance as under $\mathbb{Q}$, only a different (in general, non-zero) mean. Since $\ln P(t, T)$ from (15) is an *affine* function of $(x(t), y(t))$, it too remains Gaussian under $\mathbb{Q}^t$, and its *variance* under $\mathbb{Q}^t$ equals its variance under $\mathbb{Q}$ conditional on $\mathcal{F}_0$.

4.3 The variance $\Sigma^2(t, T)$

From (15), conditional on $\mathcal{F}_0$,

$$\ln P(t, T) = (\text{deterministic}) - B(a, \tau)x(t) - B(b, \tau)y(t), \quad \tau = T - t,$$

so, using the OU marginal variance/covariance from Section 2.2 (evaluated under $\mathbb{Q}$, and by the Girsanov argument above equally valid under $\mathbb{Q}^t$):

$$\begin{aligned} \Sigma^2(t, T) &:= \text{Var}[\ln P(t, T)] = B(a, \tau)^2 \text{Var}[x(t)] + B(b, \tau)^2 \text{Var}[y(t)] + 2B(a, \tau)B(b, \tau) \text{Cov}[x(t), y(t)] \\ &= \frac{\sigma^2}{2a^3} (1 - e^{-a\tau})^2 (1 - e^{-2at}) + \frac{\eta^2}{2b^3} (1 - e^{-b\tau})^2 (1 - e^{-2bt}) \\ &\quad + \frac{2\rho\sigma\eta}{ab(a+b)} (1 - e^{-a\tau})(1 - e^{-b\tau})(1 - e^{-(a+b)t}). \end{aligned} \quad (16)$$

(Substitute $\text{Var}[x(t)] = \frac{\sigma^2}{2a} (1 - e^{-2at})$ and $B(a, \tau)^2 = \frac{1}{a^2} (1 - e^{-a\tau})^2$ to see the first term collapses to $\frac{\sigma^2}{2a^3} (1 - e^{-a\tau})^2 (1 - e^{-2at})$; the cross term follows from (6) the same way.) Code: [G2ppAnalytics.ZeroCouponBondOption](#) (inline, as `sigmaSq`).

4.4 Closed-form price

Because $\ln P(t, T)$ is Gaussian under $\mathbb{Q}^t$ with known variance (16), and because the forward bond price $F(0; t, T) := P(0, T)/P(0, t)$ is by construction a $\mathbb{Q}^t$ -martingale (the defining property of the forward measure), the expectation $\mathbb{E}^{\mathbb{Q}^t}[(N P(t, T) - X)^+]$ is exactly the Black-76 formula applied to the lognormal forward $N \cdot F(0; t, T)$ with total (non-annualized) volatility $\Sigma(t, T) = \sqrt{\Sigma^2(t, T)}$:

$$\begin{aligned} \text{ZBC}(t, T, X, N) &= N P(0, T) \Phi(h_1) - X P(0, t) \Phi(h_2), \\ \text{ZBP}(t, T, X, N) &= X P(0, t) \Phi(-h_2) - N P(0, T) \Phi(-h_1), \\ h_1 &= \frac{1}{\Sigma} \ln \frac{N P(0, T)}{X P(0, t)} + \frac{\Sigma}{2}, \quad h_2 = h_1 - \Sigma. \end{aligned} \quad (17)$$

This is structurally identical to Jamshidian's [\[\[4\]\]](#) single-factor Hull-White bond-option formula, generalized to the two-correlated-factor variance (16); see Brigo & Mercurio [\[\[1\]\]](#), §4.2. Code: [G2ppAnalytics.ZeroCouponBondOption](#), using [AnalyticalPricers.NormalDistribution.Cdf](#) for Φ .

4.5 Edge cases and identities

- **Zero-vol limit** ($\Sigma \rightarrow 0$): the code returns the discounted intrinsic value $P(0, t) \cdot (F - X/N \cdot N)^+$ directly (bypassing $\log(\cdot)/\Sigma$, which is a $0/0$ singularity), matching the deterministic forward price $F = NP(0, T)/P(0, t)$. Verified in `ZboCallNearZeroVolMatchesDiscountedIntrinsic`.
- **Deep ITM / deep OTM**: as $X \rightarrow 0$ the call price $\rightarrow NP(0, T) - XP(0, t)$ (pure discounted forward minus discounted strike); as $X \rightarrow \infty$ it $\rightarrow 0$. Verified in `ZboDeepItmCallApproachesDiscountedForwardMinusStrike` and `ZboDeepOtmCallIsNearZero`.
- **Put-call parity** (model-free, from $(\cdot)^+ - (-\cdot)^+ = \cdot$): $ZBC - ZBP = NP(0, T) - XP(0, t)$, verified to machine precision in `ZboPutCallParityHoldsExactly`.

5 Calibration Methodology

5.1 Calibration instruments

A calibration instrument (`G2ppCalibrationInstrument`) is a tuple $(t_i, T_i, X_i, N_i, \text{isCall}_i, \text{MarketPrice}_i)$: a European zero-coupon-bond option with a known market price. In practice these would be derived from cap/floor or swaption quotes; that market-convention conversion is outside the scope of this pricer, which calibrates directly to ZBO prices.

5.2 Objective function

Given a candidate parameter vector $\theta = (a, \sigma, b, \eta, \rho)$, define the model price $\hat{P}_i(\theta)$ via (17) and minimize the mean squared *relative* pricing error:

$$\text{RMSE}(\theta)^2 = \frac{1}{n} \sum_{i=1}^n \left(\frac{\hat{P}_i(\theta) - \text{MarketPrice}_i}{\text{MarketPrice}_i} \right)^2. \quad (18)$$

Relative errors are used (rather than absolute) so that instruments of very different price scales – short-dated vs. long-dated options – contribute comparably; an absolute-error objective would be dominated by the largest premium in the instrument set. Code: `G2ppCalibrator.Calibrate`, local function `Objective`.

5.3 Unconstrained reparameterization

θ is constrained ($a, b > 0$; $\sigma, \eta \geq 0$; $\rho \in (-1, 1)$), but the optimizer (Section 5.4) is unconstrained. The search is performed over $u \in \mathbb{R}^5$ via the bijection

$$a = e^{u_1}, \quad \sigma = e^{u_2}, \quad b = e^{u_3}, \quad \eta = e^{u_4}, \quad \rho = \tanh(u_5), \quad (19)$$

so that every point visited by the simplex maps to a valid `G2ppParameters` instance; no boundary-handling logic is needed inside the objective.

5.4 Nelder-Mead simplex minimization

The optimizer is the classical derivative-free downhill simplex method of Nelder & Mead [[3]], chosen to avoid an external optimization dependency (the model has only 5 parameters and the objective is cheap to evaluate in closed form, so gradient information is unnecessary). Standard textbook coefficients are used throughout: reflection $\alpha = 1$, expansion $\gamma = 2$, contraction $\rho_c = 0.5$, shrink $\sigma_s = 0.5$.

Algorithm 1 Nelder-Mead (as implemented in `NelderMead.Minimize`)

```

1: Build an initial simplex of  $n+1$  vertices in  $\mathbb{R}^n$  from the guess  $u_0$ .
2: while iterations < max and spread of vertex values  $\geq$  tol do
3:   Sort vertices by objective value; let  $\bar{u}$  = centroid of all but the worst.
4:   Reflect:  $u_r = \bar{u} + \alpha(\bar{u} - u_{\text{worst}})$ .
5:   if  $f(u_r) < f(u_{\text{best}})$  then
6:     Expand:  $u_e = \bar{u} + \gamma(\bar{u} - u_{\text{worst}})$ ; keep the better of  $u_r, u_e$ .
7:   else if  $f(u_r) < f(u_{\text{second-worst}})$  then
8:     Accept  $u_r$ .
9:   else
10:    Contract:  $u_c = \bar{u} + \rho_c(u_{\text{worst}} - \bar{u})$ .
11:    if  $f(u_c) < f(u_{\text{worst}})$  then accept  $u_c$ 
12:    else shrink every vertex toward  $u_{\text{best}}$  by  $\sigma_s$ 
13:    end if
14:  end if
15: end while
16: return best vertex, its value, and a Converged flag.

```

5.5 Overfitting guard and failure handling

The model has 5 free parameters; `G2ppCalibrator.Calibrate` rejects (throws `ArgumentException`) any request with fewer than 5 instruments – an underdetermined least-squares fit is curve-fitting noise, not calibration (see the quant-pricing review checklist item “calibration overfitting”). After optimization, if $\text{RMSE}(\hat{\theta}) > \text{maxAcceptableRmse}$ (default 10^{-3}), the method throws `InvalidOperationException` rather than returning a poorly-fitted parameter set – consistent with the library-wide rule of *no silent fallback on calibration failure*.

5.6 Round-trip validation

The correct sanity check for a nonlinear least-squares fit to a handful of instruments is not that the raw parameter vector $\hat{\theta}$ matches a “true” generating θ bit-for-bit (the fit need not be unique), but that *the fitted model reprices the calibration instruments*. This is exactly what `G2ppCalibratorTests.Calibrate_RecoversInstrumentPrices_WithinTightTolerance` checks: synthetic ZBO prices are generated from a known θ_{true} across a 7-instrument expiry/tenor/moneyness grid, then `G2ppCalibrator.Calibrate` is run on those prices alone, and every instrument is re-priced under $\hat{\theta}$ to relative error $< 10^{-3}$.

6 Monte Carlo Simulation of the Model

6.1 Pricing identity

Under the money-market numeraire, the time-0 price of any payoff stream $\{CF_i \text{ at } t_i\}$ is

$$V(0) = \mathbb{E}^{\mathbb{Q}} \left[\sum_i \frac{CF_i}{B(t_i)} \right], \quad B(t) = \exp \left(\int_0^t r(s) ds \right). \quad (20)$$

`CallableBondMCPricer` simulates (x, y) paths exactly (Section 2.3), builds $r(t_i) = x(t_i) + y(t_i) + \varphi(t_i)$ on a fine uniform time grid, and accumulates $B(t_i)^{-1}$ path-by-path.

6.2 Fine grid and φ precomputation

Let T_M be the bond’s maturity (in years from valuation) and let n be the number of steps in the supplied `SimulationCube` (its `Steps` property); $\Delta t = T_M/n$. $\varphi(i\Delta t)$, $i = 0, \dots, n$, is

precomputed once via (14) (shared across all paths – `CallableBondMCPricer` constructor, field `.phi`).

6.3 Discounting via trapezoidal integration

The continuous integral $\int_0^t r(s) ds$ in $B(t)^{-1}$ is approximated along each simulated path by the trapezoidal rule on the fine grid:

$$B(t_i)^{-1} \approx \exp \left(- \sum_{k=1}^i \frac{1}{2} (r(t_{k-1}) + r(t_k)) \Delta t \right). \quad (21)$$

The trapezoidal rule is exact for a piecewise-linear integrand and has local error $O(\Delta t^3)$ (global $O(\Delta t^2)$) for a smooth integrand; unlike the factor simulation (Section 2.3), this step is the *only* source of time-discretization bias in the engine, and it vanishes as $\Delta t \rightarrow 0$ (Section 11 quantifies this in a deterministic setting where it can be measured exactly).

6.4 Correlated increments

The two driving Brownians in (7) are supplied by `RandomSimulator.SimulationCube.Generate(paths, steps, assets=2, correlation)`, where `correlation` = $\begin{bmatrix} 1 & \rho \\ \rho & 1 \end{bmatrix}$. The constructor of `CallableBondMCPricer` validates that `cube.Correlation[0,1]` matches `G2ppParameters.Rho` to 10^{-9} , since the cube’s own Cholesky factorization – not any code in the bond pricer – is what actually correlates Z_1, Z_2 .

7 The Callable Bond Product

7.1 Cashflow schedule

`CallableBondDefinition.BuildCashflowSchedule` generates coupon dates by stepping backward from the maturity date in $12/m$ -month increments ($m = \text{CouponsPerYear}$), retaining only dates strictly after the valuation date, using Act/365 year fractions consistent with `ZeroCurve`. The final scheduled date carries both the last coupon and the principal redemption N .

7.2 Call schedule economics

Each call date t_k^c carries a call price K_k (fraction of par). From the *bondholder’s* perspective, being called at t_k^c means receiving $K_k N$ instead of the (higher, in expectation, whenever the issuer chooses to call) remaining cashflows – i.e. the bondholder is implicitly short a Bermudan option to the issuer. Formally, if $C(t_k^c)$ denotes the market value of continuing to hold the bond at t_k^c , the issuer calls iff $K_k N < C(t_k^c)$ (calling is cheaper for the issuer than continuing to owe the future cashflows), so the bondholder’s realized value at t_k^c is

$$V(t_k^c) = \min(K_k N, C(t_k^c)). \quad (22)$$

This is a no-arbitrage bound: $V(t_k^c) \leq C(t_k^c)$ always, hence a callable bond can never be worth more than the corresponding bullet bond (verified in `CallableBond.NeverWorthMoreThanBulletBond`).

7.3 Merged event grid

`CallableBondMCPricer.BuildEventGrid` merges the coupon schedule and the call schedule into a single chronologically sorted sequence of *event nodes*, each carrying (year-fraction, cashflow amount, is-callable flag, call price). A date that is both a coupon date and a call date (the typical convention – issuers call on coupon dates) is represented by a single node with both fields

populated; this generalizes without special-casing. Each event’s year-fraction is snapshotted to the nearest fine-grid index (Section 11 discusses the resulting, intentionally small, discretization error).

8 Longstaff-Schwartz Valuation of the Embedded Call

8.1 Dynamic-programming formulation

Let $t_1 < \dots < t_M$ be the merged event dates ($t_M = T_M$, maturity). Define, for each event index k working *backward*, the bondholder’s value process V_k (a random variable indexed by simulation path) satisfying the backward recursion

$$V_M = \text{Coupon}_M + N, \quad V_k = \begin{cases} \text{Coupon}_k + \min(K_k N, C_k) & t_k \text{ callable,} \\ \text{Coupon}_k + C_k & \text{otherwise,} \end{cases} \quad (23)$$

where $C_k = \mathbb{E}^{\mathbb{Q}}[V_{k+1} \cdot B(t_k)/B(t_{k+1}) \mid \mathcal{F}_{t_k}]$ is the continuation value discounted to t_k . This is a standard optimal stopping / Bellman recursion; the price is $\mathbb{E}[V_1 \cdot B(0)/B(t_1)]$ carried all the way back to $t = 0$.

8.2 Why regression: avoiding look-ahead bias

C_k is a conditional expectation given the Markov state $(x(t_k), y(t_k))$ – it cannot be computed exactly from a finite set of simulated paths without knowing the future optimal exercise policy, which is exactly what is being determined. Longstaff & Schwartz [2]’s insight is to *estimate* C_k by least-squares regression of the (already-known, from the backward pass so far) realized continuation values on a polynomial basis in the state, and to use this regression *only* to decide whether to exercise – never as the value carried backward when not exercising, which instead uses the actual realized (simulated) continuation. This avoids the look-ahead bias that would result from using an in-sample-optimal, path-omniscient exercise rule.

8.3 Basis functions

The regression basis used is the standard low-order polynomial basis for a two-factor Gaussian short-rate model (matching the low-order polynomial recommendation of [2], §3, adapted to two state variables):

$$\psi(x, y) = (1, x, y, x^2, y^2, xy) \in \mathbb{R}^6. \quad (24)$$

At each callable event k , ordinary least squares fits $\hat{C}_k(x, y) = \beta \cdot \psi(x, y)$ to the cross-sectional sample $\{(x_p(t_k), y_p(t_k), \text{pathPV}_p)\}_{p=1}^{\text{Paths}}$ by solving the normal equations $\Psi^\top \Psi \beta = \Psi^\top v$ via Gaussian elimination with partial pivoting on the resulting 6×6 system (`CallableBondMCPricer.RegressContinuation`, `SolveLinearSystem`) – small enough that no external linear-algebra dependency is warranted.

8.4 Reformulation in PV_0 units: the implemented algorithm

Rather than carrying values forward in “value at t_k ” units and discounting stepwise (the textbook LSM presentation), the implementation carries everything in *present-value-at-0* (PV_0) units throughout, which removes an entire bookkeeping layer without changing the decision: scaling both sides of a comparison by the same positive discount factor $D(0, t_k)$ does not change which side is larger, so

$$K_k N < C_k \iff K_k N D(0, t_k) < C_k D(0, t_k),$$

i.e. the call decision can equivalently compare *PV_0 -of-call-value* against *PV_0 -of-continuation*. Let `pathPVp` accumulate the PV_0 of all cashflows from the currently-processed event onward,

under the optimal policy determined so far (i.e. at later event nodes, already processed in this backward sweep). Processing event k in decreasing order:

Algorithm 2 Backward induction at event k (per path p), as implemented in `CallableBondMCPricer.Price`

```

1: if node  $k$  is callable then
2:    $\text{contPV}_p \leftarrow \text{pathPV}_p$  ▷ continuation value excluding today's coupon
3:   Regress  $\{\text{contPV}_p\}$  on  $\psi(x_p(t_k), y_p(t_k)) \rightarrow$  coefficients  $\hat{\beta}$ 
4:   for each path  $p$  do
5:      $\hat{C}_p \leftarrow \hat{\beta} \cdot \psi(x_p(t_k), y_p(t_k))$ 
6:      $\text{callValuePV}_p \leftarrow K_k \cdot N \cdot D_p(0, t_k)$ 
7:     if  $\text{callValuePV}_p < \hat{C}_p$  then
8:        $\text{pathPV}_p \leftarrow \text{callValuePV}_p$  ▷ exercise: use the realized, not estimated, call value
9:     end if ▷ else:  $\text{pathPV}_p$  unchanged = its realized (simulated) continuation
10:  end for
11: end if
12:  $\text{pathPV}_p \leftarrow \text{pathPV}_p + \text{Coupon}_k \cdot D_p(0, t_k)$  for every path  $p$  ▷ coupon received
    unconditionally, added after the decision

```

The final price and its Monte Carlo standard error are computed from $\{\text{pathPV}_p\}_{p=1}^{\text{Paths}}$ exactly as in `MCBasePricer.BuildResult` (sample mean, sample standard error, $\pm 1.9599639845400536 \times \text{SE}$ for the 95% confidence interval).

Remark 8.1 (Coupon/call coincidence). Separating the coupon addition (unconditional, step 8) from the call decision (steps 1–7, using the *pre-coupon* continuation value) correctly handles the common convention that a called bond still pays the accrued coupon due on the call date: that coupon is received regardless of the call decision, so it must not be included on either side of the comparison in step 5 – only the call price and the value of remaining *future* cashflows compete.

9 Numerical Implementation Notes

9.1 Memory layout

Only state at event nodes is retained across the forward pass – not the full fine-grid path – via arrays `xAtEvent`, `yAtEvent`, `dAtEvent` of shape $(\text{Paths} \times \text{NumEvents})$. `NumEvents` is small (coupon frequency $\times$ years, plus call dates), so this is $O(\text{Paths} \times \text{NumEvents})$ rather than $O(\text{Paths} \times \text{Steps})$, letting the fine grid (Section 6.3) be arbitrarily refined for discounting accuracy without inflating memory.

9.2 Parallelism

The forward simulation pass (`SimulateForward`) uses `Parallel.For` over paths, matching the concurrency pattern used throughout the library (`MCBasePricer`); each path writes only to its own row of the event-snapshot matrices, so no synchronization is needed. The backward LSM pass is inherently sequential across events (each regression depends on the previous event's decisions) but parallel across paths within each event.

9.3 DV01

`CallableBondMCPricer.ComputeDv01` bumps the discount curve by $\pm \epsilon$ (default 1bp) via `ZeroCurve.Shift` and reprices on the *same* `SimulationCube` (common random numbers reduce

finite-difference variance, matching the Greek methodology used by every other pricer in the library, e.g. `MCBasePricer.ComputeGreeks`). Because $\varphi(t)$ in (14) depends on the curve, bumping the curve correctly perturbs *both* discounting and the model drift.

10 Validation and Benchmarks

Check	What it validates	Test
Curve reproduction	$P(0, T) = P^M(0, T)$ exactly, any θ	<code>§3.5, G2ppAnalyticsTests</code>
Put-call parity	Model-free ZBO identity, to machine precision	<code>Zbo.PutCallParity_HoldsExactly</code>
Zero-vol / deep ITM / deep OTM	ZBO collapses to discounted intrinsic	<code>ZboCall_NearZeroVol...</code> , <code>Zbo_Deep*</code>
Calibration round-trip	Fitted $\hat{\theta}$ reprices calibration instruments	<code>Calibrate_RecoversInstrumentPrices_...</code>
Bullet-bond benchmark	MC price $\approx$ closed-form curve-discount sum	<code>NoCallSchedule_Matches...</code> , <code>CallPriceNeverReached_Matches...</code>
No-arbitrage bound	Callable price $\leq$ bullet price	<code>CallableBond_NeverWorthMoreThanBulletBond</code>
Deterministic zero-vol trace	LSM decision correct with zero MC noise	<code>ZeroVol_DeepItmCall...</code> , <code>ZeroVol_NoCallSchedule...</code>
DV01 sign	Fixed-coupon bond value falls as rates rise	<code>Dv01_IsNegative_ForFixedCouponBond</code>
Input validation	Cube shape, correlation match, date ordering	<code>Constructor_Rejects*</code> , <code>CallableBondDefinition_Rejects*</code>

All 32 tests in `MCPricer.Tests/Bonds` pass with the deterministic seed convention used throughout the library (seed = 42).

11 Known Limitations

- **LSM in-sample bias.** Using the same simulated paths for both the continuation regression and the final valuation is known to introduce a small, generally upward, bias in American/Bermudan-style LSM prices – the regression has effectively “seen” the payoff it is trying to predict. This is not corrected here; an (asymptotically) bias-free estimate would require an independent out-of-sample path set for the final valuation pass, at roughly double the simulation cost. Documented in the class-level comment of `CallableBondMCPricer`.
- **Event-to-grid rounding bias.** Coupon dates are Act/365 calendar-day-count year fractions that in general do not land exactly on the uniform fine simulation grid; `BuildEventGrid` snapshots each event at its *nearest* grid node, an $O(\Delta t)$ discretization error. In a controlled deterministic (zero-vol) test this measures at $\approx 2 \times 10^{-3}$ absolute (on a par-100 bond) with a 12-steps-per-year grid, and shrinks to $< 10^{-3}$ with a daily grid – consistent with an $O(\Delta t)$ error that vanishes as the grid is refined. This is an intentional engineering trade-off (uniform grid vs. exact per-event alignment) documented in `CallableBondMCPricerTests`.
- **Trapezoidal discounting bias.** Section 6.3’s rule is exact only for a piecewise-linear short-rate path; on a flat curve with $\sigma = \eta = 0$ (so $r(t) \equiv \varphi(t) \equiv \text{const}$) it is exact identically, since the true integral of a constant is trivially linear. For non-trivial σ, η , the

bias is $O(\Delta t^2)$ globally and is not separately isolated from Monte Carlo sampling noise in the current test suite (it is dominated by MC standard error at the path counts used).

12 Code-to-Formula Mapping

File / Member	Formula	Section
G2ppParameters.cs	$(a, \sigma, b, \eta, \rho)$, validated	2
G2ppAnalytics.InstantaneousForward	$f^M(a, t)$ (finite difference)	3.4
G2ppAnalytics.Phi	$\varphi(t)$, eq. (14)	3.4
G2ppAnalytics.VFunction (private)	$V(\tau)$, eq. (11)	3.3
G2ppAnalytics.ZeroCouponBondPrice	$P(a, T)$, eq. (15)	3.5
G2ppAnalytics.ZeroCouponBondOption	ZBOn, eq. (17)	4
G2ppCalibrationInstrument	Calibration target tuple	5
G2ppCalibrator.Calibrate	RMSE objective eq. (18), reparam. eq. (19)	5
NelderMead.Minimize	Downhill simplex	5.4
CallableBondDefinition. BuildCashflowSchedule	Coupon schedule generation	7
CallableBondMCPricer ctor	Curve/cube validation, φ precompute	6
CallableBondMCPricer. SimulateForward	Exact OU step eq. (7); trapezoidal $B(t)^{-1}$ eq. (21)	2.3, 6.3
CallableBondMCPricer. BuildEventGrid	Merged coupon/call event timeline	7
CallableBondMCPricer. RegressContinuation	OLS on basis eq. (24)	8
CallableBondMCPricer. SolveLinearSystem	Gaussian elimination (partial pivot)	8
CallableBondMCPricer.Price	Backward induction eq. (23), §8.4	8
CallableBondMCPricer. AnalyticalBulletBondPrice	Model-independent benchmark	10
CallableBondMCPricer. ComputeDv01	Curve-bump Greek	9

References

- [1] Brigo, D. and Mercurio, F. (2006). *Interest Rate Models – Theory and Practice*, 2nd ed. Springer Finance. §3 (one-factor Hull-White / Vasicek) and §4.2 (the two-factor additive Gaussian model, G2++): dynamics, the curve-fitting shift $\varphi(t)$, the zero-coupon bond price, and the option-on-a-zero-coupon-bond formula.
- [2] Longstaff, F.A. and Schwartz, E.S. (2001). “Valuing American options by simulation: a simple least-squares approach.” *Review of Financial Studies* 14(1): 113–147.
- [3] Nelder, J.A. and Mead, R. (1965). “A simplex method for function minimization.” *The Computer Journal* 7(4): 308–313.
- [4] Jamshidian, F. (1989). “An exact bond option formula.” *The Journal of Finance* 44(1): 205–209.
- [5] Vasicek, O. (1977). “An equilibrium characterization of the term structure.” *Journal of*

Financial Economics 5(2): 177–188.

- [6] Hull, J. and White, A. (1994). “Numerical procedures for implementing term structure models I: single-factor models.” *Journal of Derivatives* 2(1): 7–16.